

RAG — Guide complet

Pipeline, code Python, et toutes les méthodes d'évaluation
(récupération, génération, LLM-juge) — avec RAGAS et optimisation

10 juin 2026

Table des matières

1	Pourquoi le RAG ?	2
2	Architecture et fonctionnement	2
3	Embeddings : la fondation	3
4	Mesures de similarité	3
5	Recherche : du kNN exact à l'approché (ANN)	3
6	Chunking	3
7	Découpage en chunks : les text splitters de LangChain	3
7.1	CharacterTextSplitter	4
7.2	RecursiveCharacterTextSplitter (le choix par défaut)	4
7.3	TokenTextSplitter	4
7.4	Splitters structurés : code et Markdown	4
7.5	SemanticChunker (découpage sémantique)	5
7.6	Comment choisir	5
8	Le vector store : ChromaDB en pratique	6
8.1	API native : le module chromadb	6
8.2	Wrapper LangChain : langchain_chroma	6
9	Génération : formulation probabiliste	8
10	Code : le pipeline complet (cas d'utilisation)	8
11	Le triangle du RAG : quoi évaluer	9
12	Construire le jeu d'évaluation	9
13	Métriques de récupération (top-k), en détail	10
13.1	Precision@ k , Recall@ k , F1@ k	10
13.2	Hit Rate@ k , MRR	10
13.3	MAP (Mean Average Precision)	10
13.4	NDCG@ k	11
13.5	Code : calculer ces métriques	11
14	Métriques de génération : classiques	11
15	Métriques de génération : LLM comme juge	12

16 RAGAS (contenu supplémentaire approfondi)	12
16.1 Faithfulness (fidélité) — arête $\mathcal{C} \leftrightarrow a$	13
16.2 Answer Relevancy — arête $q \leftrightarrow a$	13
16.3 Context Precision — arête $q \leftrightarrow \mathcal{C}$	13
16.4 Context Recall — arête $q \leftrightarrow \mathcal{C}$ (avec référence)	13
16.5 Answer Correctness (avec référence)	13
16.6 Autres & limites	14
16.7 Code : RAGAS de bout en bout	14
17 Tableau de diagnostic	14
18 Leviers d'optimisation (reliés aux métriques)	14
19 Méthodologie : la boucle fermée	16
20 Synthèse générale	16

Partie I — Le pipeline RAG

1 Pourquoi le RAG ?

Un LLM a une connaissance **figée**, ignore vos **données privées**, et peut **halluciner**. Le **RAG** (génération augmentée par récupération) **récupère** des documents pertinents puis les **injecte** dans le prompt pour ancrer la réponse. Bénéfices : à jour (on met à jour la base, pas le modèle), traçable (citations $\Rightarrow$ moins d'hallucinations), privé, et moins cher qu'un fine-tuning.

2 Architecture et fonctionnement

Phase 1 — Indexation (hors ligne) : charger $\rightarrow$ découper (chunking) $\rightarrow$ encoder (embeddings) $\rightarrow$ stocker (base vectorielle).

Phase 2 — Récupération + génération (en ligne) : encoder la question $\rightarrow$ rechercher le top- k $\rightarrow$ augmenter le prompt $\rightarrow$ générer.

Idée principale

Le RAG transforme un problème *linguistique* (comprendre le sens) en un problème *géométrique* (mesurer des distances entre vecteurs), que l'on sait traiter à grande échelle. Tout le pipeline découle de cette idée.

3 Embeddings : la fondation

Définition 3.1 (Fonction d'embedding). $E : \mathcal{T} \rightarrow \mathbb{R}^d$ envoie un texte vers un vecteur dense, de sorte que la **proximité géométrique** reflète la **proximité sémantique**.

Intuition

Espace *dense* ($d \sim$ centaines/milliers, coordonnées non nulles) : capture synonymes et contexte.
Espace *creux* (sac de mots) : capture les mots exacts. Le RAG moderne combine souvent les deux.

4 Mesures de similarité

Soit $\mathbf{q} = E(q)$, $\mathbf{c} = E(c)$.

$$\langle \mathbf{q}, \mathbf{c} \rangle = \mathbf{q}^\top \mathbf{c}, \quad \text{sim}_{\cos}(\mathbf{q}, \mathbf{c}) = \frac{\mathbf{q}^\top \mathbf{c}}{\|\mathbf{q}\| \|\mathbf{c}\|} = \cos \theta, \quad \|\mathbf{q} - \mathbf{c}\|_2 = \sqrt{\sum_i (q_i - c_i)^2}.$$

Proposition 4.1 (Vecteurs normalisés). Si $\|\mathbf{q}\| = \|\mathbf{c}\| = 1$: $\|\mathbf{q} - \mathbf{c}\|_2^2 = 2 - 2 \text{sim}_{\cos}(\mathbf{q}, \mathbf{c})$.

Démonstration. $\|\mathbf{q} - \mathbf{c}\|^2 = \|\mathbf{q}\|^2 + \|\mathbf{c}\|^2 - 2\mathbf{q}^\top \mathbf{c} = 2 - 2 \cos \theta$. □

Idée principale

Sur vecteurs **normalisés**, minimiser la distance euclidienne = maximiser le cosinus (même classement). On normalise donc les embeddings, et on préfère le cosinus au produit scalaire brut pour qu'un texte long (grande norme) ne soit pas favorisé artificiellement.

5 Recherche : du kNN exact à l'approché (ANN)

Le top- k exact $\text{top-k}(q) = \arg \max_{|S|=k} \sum_{j \in S} \text{sim}(\mathbf{q}, \mathbf{c}_j)$ coûte $O(Nd)$ par requête. Pour $N \sim$ millions, on utilise un index **ANN** : **HNSW** (graphe, $O(\log N)$), **IVF** (cellules/clustering), **PQ** (quantification/compression).

Idée principale

On échange un peu d'exactitude contre beaucoup de vitesse. Le *recall de l'index* (fraction des vrais voisins retrouvés) est une **première source d'erreur**, en amont des embeddings.

6 Chunking

Découper en segments de longueur $\approx L$ avec recouvrement $o : \approx |D|/(L - o)$ chunks.

Idée principale

Le chunk est l'**unité atomique** de récupération. L grand $\Rightarrow$ embedding qui « moyenne » plusieurs idées (recherche floue) ; L petit $\Rightarrow$ contexte tronqué. Le recouvrement o évite de couper une idée à la frontière. On règle L, o *empiriquement*, via le context recall.

7 Découpage en chunks : les text splitters de LangChain

En pratique, on choisit *comment* couper. LangChain fournit plusieurs stratégies, du plus simple au plus sémantique.

7.1 CharacterTextSplitter

Coupe sur **un seul séparateur** (p. ex. `"\n\n"`), puis regroupe les morceaux jusqu'à atteindre `chunk_size`.

Listing 1 – CharacterTextSplitter

```
1 from langchain.text_splitter import CharacterTextSplitter
2 splitter = CharacterTextSplitter(
3     separator="\n\n", chunk_size=800, chunk_overlap=100)
4 chunks = splitter.split_text(texte)           # liste de str
5 docs = splitter.create_documents([texte])      # liste de Document
```

Intuition

Simple mais rigide : si un paragraphe dépasse `chunk_size`, il n'est pas re-coupé finement. À réserver aux textes bien structurés.

7.2 RecursiveCharacterTextSplitter (le choix par défaut)

Essaie une **hiérarchie de séparateurs** `["\n\n", "\n", " ", ""]` **récurivement** : il coupe d'abord par paragraphes ; si un morceau reste trop grand, il le recoupe par lignes, puis par mots, puis par caractères.

Listing 2 – RecursiveCharacterTextSplitter

```
1 from langchain.text_splitter import RecursiveCharacterTextSplitter
2 splitter = RecursiveCharacterTextSplitter(
3     chunk_size=800, chunk_overlap=100,
4     separators=["\n\n", "\n", " ", ""], # ordre de priorité
5     length_function=len)                # ou un compteur de tokens
6 chunks = splitter.split_documents(docs)
```

Idée principale

Idée centrale : **respecter la structure naturelle** du texte le plus longtemps possible. En coupant d'abord aux frontières « fortes » (paragraphes) et seulement ensuite aux frontières « faibles », on garde des unités sémantiquement cohérentes. C'est pour cela que c'est le splitter recommandé par défaut.

7.3 TokenTextSplitter

Découpe en comptant les **tokens** (via `tiktoken`) plutôt que les caractères.

Listing 3 – TokenTextSplitter

```
1 from langchain.text_splitter import TokenTextSplitter
2 splitter = TokenTextSplitter(chunk_size=256, chunk_overlap=32)
3 chunks = splitter.split_text(texte)
```

Pourquoi ça aide

Les LLM et les modèles d'embedding ont une limite **en tokens**, pas en caractères. Compter en tokens garantit qu'aucun chunk ne dépasse la fenêtre du modèle ⇒ pas de troncature silencieuse.

7.4 Splitters structurés : code et Markdown

`RecursiveCharacterTextSplitter.from_language` utilise des séparateurs adaptés à un langage (fonctions, classes...); `MarkdownHeaderTextSplitter` coupe selon les titres et garde la hiérarchie en métadonnées.

Listing 4 – Code-aware et Markdown

```
1 from langchain.text_splitter import (RecursiveCharacterTextSplitter,
2                                     Language, MarkdownHeaderTextSplitter)
3 py = RecursiveCharacterTextSplitter.from_language(
4     language=Language.PYTHON, chunk_size=500, chunk_overlap=0)
5
6 md = MarkdownHeaderTextSplitter(headers_to_split_on=[
7     ("#", "h1"), ("##", "h2"), ("###", "h3")])
8 sections = md.split_text(markdown_text) # chaque chunk porte ses titres
```

Pourquoi ça aide

Découper selon la **structure** (fonction, section) plutôt qu'une longueur arbitraire évite de couper au milieu d'une idée, et les métadonnées de titre permettent un **filtrage** fin à la recherche.

7.5 SemanticChunker (découpage sémantique)

Au lieu d'une longueur fixe, il coupe là où le **sens change**. Fonctionnement :

1. découper le texte en **phrases** ;
2. **encoder** chaque phrase (embeddings) ;
3. calculer la **distance cosinus** entre phrases *consécutives* ;
4. placer un **point de coupure** là où la distance dépasse un seuil (p. ex. le 95^e percentile) ;
5. regrouper les phrases entre deux coupures en un chunk.

Listing 5 – SemanticChunker

```
1 from langchain_experimental.text_splitter import SemanticChunker
2 from langchain_huggingface import HuggingFaceEmbeddings
3 emb = HuggingFaceEmbeddings(
4     model_name="sentence-transformers/all-MiniLM-L6-v2")
5 splitter = SemanticChunker(
6     emb, breakpoint_threshold_type="percentile" # ou standard_deviation,
7         gradient
8 )
9 chunks = splitter.create_documents([texte])
```

Idée principale

Idée : les frontières de chunk doivent tomber aux **ruptures de sujet**, pas à un nombre fixe de caractères. Une distance élevée entre deux phrases consécutives signale un changement de thème $\Rightarrow$ on coupe là. Résultat : des chunks très **cohérents** (embeddings plus « purs », $\uparrow$ context precision), au prix d'un coût de calcul plus élevé (il faut encoder toutes les phrases).

7.6 Comment choisir

Splitter	Quand	Coût
Recursive (caractères)	défaut, texte général	faible
Token	respecter la limite du modèle	faible
from_language / Markdown	code, docs structurés	faible
Semantic	cohérence maximale des chunks	élevé (embeddings)

8 Le vector store : ChromaDB en pratique

Chroma stocke les vecteurs et fait la recherche des plus proches voisins. Deux niveaux d'API : le module `chromadb` (client natif, contrôle fin) et le wrapper `langchain_chroma` (intégré au pipeline LangChain).

8.1 API native : le module chromadb

Initialisation. Client en mémoire ou **persistant** (sauvegarde sur disque) :

Listing 6 – chromadb : client, collection

```
1 import chromadb
2 # Persistant : les données survivent au redémarrage
```

```

3 client = chromadb.PersistentClient(path="./chroma_db")
4 # (ou chromadb.Client() pour un client ephemere en memoire)
5
6 # Une "collection" = une table de vecteurs. hnsw:space fixe la distance.
7 col = client.get_or_create_collection(
8     name="rag_docs", metadata={"hnsw:space": "cosine"}) # cosine / l2 / ip

```

Ajouter des documents (add). On fournit textes, **ids** (obligatoires), métadonnées, et éventuellement les embeddings (sinon Chroma les calcule avec sa fonction d'embedding) :

Listing 7 – chromadb : add / upsert

```

1 col.add(
2     ids=["d1", "d2", "d3"],
3     documents=["Paris est la capitale.", "Lyon est une ville.",
4               "La Seine traverse Paris."],
5     metadatas=[{"source": "geo.pdf", "page": 1},
6                 {"source": "geo.pdf", "page": 2},
7                 {"source": "geo.pdf", "page": 3}],
8     # embeddings=[...], [...], [...]] # optionnel si fournis a la main
9 )
10 col.upsert(ids=["d1"], documents=["Paris, capitale de la France."]) # ajoute
    ou met a jour

```

Listing 8 – chromadb : query, filters, get/update/delete

Interroger (query), filtrer, gérer.

```

1 # Recherche top-k, avec filtre sur metadonnees (where) et sur texte
  (where_document)
2 res = col.query(
3     query_texts=["Quelle est la capitale ?"], n_results=2,
4     where={"source": "geo.pdf"},
5     where_document={"$contains": "capitale"})
6 print(res["documents"], res["distances"])
7
8 col.get(ids=["d1"]) # recuperer par id
9 col.get(where={"page": {"$gte": 2}}) # filtrer
  ($eq, $ne, $gt, $gte, $lt, $in, $and, $or)
10 col.update(ids=["d2"], metadatas=[{"source": "geo_v2.pdf"}])
11 col.delete(ids=["d3"]) # ou delete(where={...})
12 print(col.count()) # nombre de vecteurs

```

8.2 Wrapper LangChain : langchain_chroma

Plus pratique pour le RAG : il branche directement un modèle d'embedding et expose un retriever.

Listing 9 – langchain_chroma : creer / charger

Initialisation (3 cas).

```

1 from langchain_chroma import Chroma
2 from langchain_huggingface import HuggingFaceEmbeddings
3 emb = HuggingFaceEmbeddings(
4     model_name="sentence-transformers/all-MiniLM-L6-v2")
5
6 # (a) construire depuis des documents (encode + stocke)
7 vs = Chroma.from_documents(
8     documents=chunks, embedding=emb,
9     collection_name="rag_docs", persist_directory="./chroma_db",
10    collection_metadata={"hnsw:space": "cosine"})
11

```

```

12 # (b) construire depuis des textes bruts
13 vs = Chroma.from_texts(texts=["...", "..."], embedding=emb,
14     metadatas=[{"source": "a"}, {"source": "b"}], ids=["t1", "t2"])
15
16 # (c) RECHARGER une base existante (sans re-encoder)
17 vs = Chroma(collection_name="rag_docs", embedding_function=emb,
18     persist_directory="./chroma_db")

```

Listing 10 – langchain_chroma : add_documents, search, delete
Ajouter, rechercher, filtrer, supprimer.

```

1 # Ajouter de nouveaux documents (avec ids pour pouvoir mettre a jour)
2 vs.add_documents(documents=nouveaux_chunks, ids=["x1", "x2"])
3 vs.add_texts(texts=["..."], metadatas=[{"source": "c"}], ids=["t3"])
4
5 # Recherche par similarite (top-4)
6 docs = vs.similarity_search("capitale de la France", k=4)
7
8 # ... avec le score (distance : plus petit = plus proche)
9 pairs = vs.similarity_search_with_score("capitale", k=4)
10
11 # ... avec filtre sur metadonnees
12 docs = vs.similarity_search("ville", k=4, filter={"source": "geo.pdf"})
13
14 # Recherche par vecteur deja calcule
15 docs = vs.similarity_search_by_vector(emb.embed_query("Paris"), k=4)
16
17 # Diversite (MMR) : recupere fetch_k puis garde k diversifies
18 docs = vs.max_marginal_relevance_search("Paris", k=4, fetch_k=20,
19     lambda_mult=0.5)
20
21 vs.delete(ids=["x1"]) # supprimer par id

```

Listing 11 – langchain_chroma : as_retriever
Transformer en retrieveur (pour la chaîne RAG).

```

1 # Similarite simple
2 retriever = vs.as_retriever(search_kwargs={"k": 4})
3
4 # MMR (diversite)
5 retriever = vs.as_retriever(
6     search_type="mmr",
7     search_kwargs={"k": 4, "fetch_k": 20, "lambda_mult": 0.5})
8
9 # Seuil de score (ne garder que les chunks assez proches)
10 retriever = vs.as_retriever(
11     search_type="similarity_score_threshold",
12     search_kwargs={"score_threshold": 0.5, "k": 4})
13
14 # Avec filtre metadonnees
15 retriever = vs.as_retriever(
16     search_kwargs={"k": 4, "filter": {"source": "geo.pdf"}})

```

Idée principale

Le **retriever** est le **pont** entre le vector store et la chaîne RAG : c'est l'objet qu'on branche dans le pipeline (**retriever** | **prompt** | **llm**). Les options (**k**, **mmr**, **filter**, **score_threshold**) sont autant de **leviers d'optimisation** de la récupération, mesurables via context precision/-recall.

Native ou LangChain ?

On utilise `chromadb` natif pour un contrôle fin (gestion d'ids, métadonnées complexes, scripts d'ingestion) et `langchain_chroma` pour brancher directement la base dans une chaîne RAG. Les deux écrivent dans le *même* format sur disque : avec `PersistentClient/persist_directory`, la base est sauvegardée automatiquement (les versions récentes n'exigent plus d'appel `.persist()`). Pensez à fixer la distance à `cosine` et à **normaliser** vos embeddings pour la cohérence.

9 Génération : formulation probabiliste

Le LLM génère token par token, conditionné sur question et contexte :

$$P_{\theta}(y \mid q, c) = \prod_{t=1}^T P_{\theta}(y_t \mid y_{<t}, q, c).$$

Comme le retrieveur renvoie k documents, le modèle RAG **marginalise** sur le document latent c , pondéré par la probabilité de récupération $P_{\eta}(c \mid q) = \text{softmax}(\text{sim}(\mathbf{q}, \mathbf{c})/\tau)$:

$$\textbf{RAG-Sequence} : P(y \mid q) \approx \sum_{c \in \text{top-}k(q)} P_{\eta}(c \mid q) P_{\theta}(y \mid q, c),$$

$$\textbf{RAG-Token} : P(y \mid q) \approx \prod_t \sum_{c \in \text{top-}k(q)} P_{\eta}(c \mid q) P_{\theta}(y_t \mid y_{<t}, q, c).$$

Idée principale

La réponse est conditionnée à une **moyenne pondérée** des k documents, chaque poids étant la confiance de récupération. En pratique, on *concatène* les chunks dans le prompt : une approximation où le LLM pondère implicitement.

10 Code : le pipeline complet (cas d'utilisation)

Stack locale : `langchain` (orchestration), `chromadb` (base vectorielle), `sentence-transformers` (embeddings), `ollama` (LLM local, Llama 3.1).

Listing 12 – Installation

```
1 pip install langchain langchain-community langchain-chroma \
2   langchain-huggingface langchain-ollama \
3   chromadb sentence-transformers pypdf
4 # ollama pull llama3.1
```

Listing 13 – Indexation : charger -> découper -> encoder -> stocker

```
1 from langchain_community.document_loaders import PyPDFLoader
2 from langchain.text_splitter import RecursiveCharacterTextSplitter
3 from langchain_huggingface import HuggingFaceEmbeddings
4 from langchain_chroma import Chroma
5
6 docs = PyPDFLoader("mon_document.pdf").load()
7 chunks = RecursiveCharacterTextSplitter(
8     chunk_size=800, chunk_overlap=100).split_documents(docs)
9 emb = HuggingFaceEmbeddings(
10     model_name="sentence-transformers/all-MiniLM-L6-v2")
11 vs = Chroma.from_documents(chunks, emb, persist_directory="./chroma_db")
```


Listing 14 – Recuperation + generation (LCEL)

```

1 from langchain_ollama import ChatOllama
2 from langchain_core.prompts import ChatPromptTemplate
3 from langchain_core.runnables import RunnablePassthrough
4 from langchain_core.output_parsers import StrOutputParser
5
6 retriever = vs.as_retriever(search_kwargs={"k": 4})
7 llm = ChatOllama(model="llama3.1", temperature=0)
8 prompt = ChatPromptTemplate.from_template(
9     "Reponds UNIQUEMENT a partir du contexte. Si absent, dis-le.\n"
10     "Contexte:\n{context}\n\nQuestion: {question}\nReponse:")
11
12 def fmt(ds): return "\n\n".join(d.page_content for d in ds)
13
14 chain = ({ "context": retriever | fmt, "question": RunnablePassthrough() }
15         | prompt | llm | StrOutputParser())
16 print(chain.invoke("Quelle est la conclusion du document ?"))

```

Partie II — Évaluation complète

11 Le triangle du RAG : quoi évaluer

On évalue les **trois arêtes** du triangle (question q , contexte $\mathcal{C}$, réponse a) :

Arête	Question	Maillon
$q \leftrightarrow \mathcal{C}$	Contexte pertinent ?	Récupération
$\mathcal{C} \leftrightarrow a$	Réponse fondée sur le contexte ?	Génération (fidélité)
$q \leftrightarrow a$	Réponse répond à la question ?	Génération (pertinence)

Idée principale

Évaluer = **localiser la panne**. Mauvaise réponse = mauvais contexte (arête $q-\mathcal{C}$) ou mauvaise exploitation d'un bon contexte (arête $\mathcal{C}-a$) ? Chaque famille de métriques répond à une arête et **oriente** l'optimisation.

12 Construire le jeu d'évaluation

Un échantillon : $(q, a \text{ (générée)}, \mathcal{C} \text{ (contextes)}, g \text{ (référence)})$. **Manuel** (fiable, coûteux) ou **synthétique** (un LLM génère les questions depuis les chunks). Distinction clé : métriques **avec référence** (context recall, correctness) vs **sans référence** (faithfulness, answer relevancy, context precision : utilisables en production).

13 Métriques de récupération (top- k), en détail

On note R_q l'ensemble des documents pertinents (vérité terrain) et $\text{rel}(i) = 1$ si le document au rang i est pertinent.

Exemple fil rouge. 5 documents récupérés, $|R_q| = 4$, pertinences sur la liste : $(1, 0, 1, 0, 1)$.

13.1 Precision@k, Recall@k, F1@k

$$P@k = \frac{\# \text{pertinents dans top-}k}{k}, \quad R@k = \frac{\# \text{pertinents dans top-}k}{|R_q|}, \quad F1@k = \frac{2 P@k R@k}{P@k + R@k}.$$

Calcul étape par étape

k	cumul	$P@k$	$R@k$ ($ R_q =4$)
1	1	1,00	0,25
3	2	0,67	0,50
5	3	0,60	0,75
$F1@5 = \frac{2 \cdot 0,60 \cdot 0,75}{0,60 + 0,75} = \frac{0,90}{1,35} \approx 0,667.$			

Intuition

$P@k$: part utile de ce qu'on sort ; $R@k$: part de l'utile retrouvée. Quand k croît, R monte, P baisse : **compromis précision/rappel**.

13.2 Hit Rate@k, MRR

$$\text{HitRate@}k = \frac{1}{|Q|} \sum_q \mathbb{I}[\exists i \leq k : \text{rel}(i) = 1], \quad \text{MRR} = \frac{1}{|Q|} \sum_q \frac{1}{\text{rang du 1}^{\text{er}} \text{ pertinent}}.$$

Exemple : premier pertinent au rang 1 $\Rightarrow$ $RR = 1$. Avec une 2^e requête (1^{er} pertinent rang 3) : $\text{MRR} = \frac{1}{2}(1 + \frac{1}{3}) = \frac{2}{3}$.

13.3 MAP (Mean Average Precision)

$$AP = \frac{1}{|R_q|} \sum_k P@k \cdot \text{rel}(k), \quad \text{MAP} = \frac{1}{|Q|} \sum_q AP_q.$$

Calcul étape par étape

Pertinents aux rangs 1, 3, 5, $|R_q| = 4$:

$$AP = \frac{1}{4} \left(\underbrace{1}_{P@1} + \underbrace{0,667}_{P@3} + \underbrace{0,600}_{P@5} \right) = \frac{2,267}{4} \approx 0,567.$$

Le pertinent **manqué** contribue 0 : la MAP pénalise rangs *et* oublis.

13.4 NDCG@k

$$\text{DCG@}k = \sum_{i=1}^k \frac{2^{\text{rel}_i} - 1}{\log_2(i+1)} \quad \left(\text{binaire : } \sum \frac{\text{rel}_i}{\log_2(i+1)} \right), \quad \text{NDCG@}k = \frac{\text{DCG@}k}{\text{IDCG@}k}.$$

Calcul étape par étape

$$\text{DCG@5} = \frac{1}{\log_2 2} + \frac{1}{\log_2 4} + \frac{1}{\log_2 6} = 1 + 0,5 + 0,387 = 1,887.$$

$$\text{Ordre idéal (3 pertinents en tête) : IDCG@5} = 1 + 0,631 + 0,5 = 2,131.$$

$$\text{NDCG@5} = 1,887 / 2,131 \approx 0,885.$$

Idée principale

Le facteur $1/\log_2(i+1)$ encode « un bon doc au rang 1 vaut plus qu'au rang 10 » ; la normalisation par l'IDCG ramène dans $[0, 1]$. La version graduée gère des pertinences « un peu/beaucoup/parfait ».

13.5 Code : calculer ces métriques

Listing 15 – Metriques de recuperation en Python pur

```
1 import numpy as np
2
3 def precision_at_k(rel, k): return sum(rel[:k]) / k
4 def recall_at_k(rel, k, R): return sum(rel[:k]) / R
5
6 def reciprocal_rank(rel):
7     for i, r in enumerate(rel, 1):
8         if r: return 1.0 / i
9     return 0.0
10
11 def average_precision(rel, R):
12     score, hits = 0.0, 0
13     for i, r in enumerate(rel, 1):
14         if r:
15             hits += 1
16             score += hits / i      # = Precision@i quand rel(i)=1
17     return score / R
18
19 def dcg_at_k(rel, k):
20     return sum(r / np.log2(i + 1) for i, r in enumerate(rel[:k], 1))
21
22 def ndcg_at_k(rel, k):
23     ideal = dcg_at_k(sorted(rel, reverse=True), k)
24     return dcg_at_k(rel, k) / ideal if ideal else 0.0
25
26 rel = [1, 0, 1, 0, 1]; R = 4
27 print(precision_at_k(rel, 5), recall_at_k(rel, 5, R))    # 0.6  0.75
28 print(average_precision(rel, R))                        # ~0.567
29 print(ndcg_at_k(rel, 5))                                # ~0.885
```

14 Métriques de génération : classiques

Comparaison de la réponse a à une référence g par chevauchement de texte.

- **ROUGE-L** (plus longue sous-séquence commune, LCS) : $R = \frac{LCS}{|g|}$, $P = \frac{LCS}{|a|}$, $F = \frac{(1+\beta^2)PR}{R+\beta^2P}$.
- **BLEU** : $BLEU = BP \cdot \exp(\sum_n w_n \log p_n)$, p_n précision n -grammes, BP pénalité de brièveté.
- **BERTScore** : appariement glouton entre embeddings de tokens, puis P/R/F sur les cosinus.
- **Token-F1** / **Exact Match** (style SQuAD) : recouvrement de tokens / égalité exacte.

Intuition

Rapides et reproductibles, mais **myopes** : elles récompensent les *mots communs*, pas la *vérité*. Une bonne réponse reformulée a un ROUGE faible ; une fausse réponse qui copie le vocabulaire a un ROUGE élevé. D'où le passage aux méthodes à base de LLM.

Listing 16 – ROUGE / BLEU / BERTScore via bibliotheques

```
1 from rouge_score import rouge_scorer
2 from sacrebleu import sentence_bleu
```

```

3 from bert_score import score as bertscore
4
5 ref = "Paris est la capitale de la France."
6 hyp = "La capitale de la France est Paris."
7
8 rl = rouge_scorer.RougeScorer(["rougeL"], use_stemmer=True)
9 print(rl.score(ref, hyp)["rougeL"].fmeasure)      # F-mesure ROUGE-L
10 print(sentence_bleu(hyp, [ref]).score)           # BLEU
11 P, R, F = bertscore([hyp], [ref], lang="fr")
12 print(F.item())                                  # F BERTScore

```

15 Métriques de génération : LLM comme juge

Un LLM puissant note la réponse selon une consigne : **note directe** (échelle 1–5 avec grille), ou **comparaison par paires** (quel système est meilleur ?).

Biais et parades

Biais : *position* (favorise la 1^{re} réponse), *verbosité* (favorise le long), *auto-préférence*. Parades : permuter l'ordre et moyenner, fournir une **grille** explicite, demander un **raisonnement** avant la note, calibrer sur quelques exemples humains.

Listing 17 – LLM-juge : note directe avec grille

```

1 JUDGE = """Tu es un évaluateur. Note de 1 à 5 la REPONSE selon :
2 - exactitude vis-a-vis du CONTEXTE
3 - pertinence vis-a-vis de la QUESTION
4 Réponds au format JSON: {"note": <1-5>, "raison": "<...>"}.
5
6 QUESTION: {q}
7 CONTEXTE: {ctx}
8 REPONSE: {a}"""
9
10 def llm_judge(llm, q, ctx, a):
11     out = llm.invoke(JUDGE.format(q=q, ctx=ctx, a=a))
12     return out.content      # parser le JSON ensuite

```

16 RAGAS (contenu supplémentaire approfondi)

RAGAS automatise l'évaluation de bout en bout via un **LLM-juge** et des embeddings, souvent *sans référence*. Principe : **décomposer** en unités atomiques, puis **vérifier/comparer**, puis agréger dans $[0, 1]$.

16.1 Faithfulness (fidélité) — arête $C \leftrightarrow a$

LLM décompose a en affirmations $\{s_1, \dots, s_n\}$, verdict $v_i \in \{0, 1\}$ (inférable du contexte) :

$$\text{Faithfulness} = \frac{1}{n} \sum_{i=1}^n v_i.$$

Calcul étape par étape

a : « Paris est la capitale et compte 10 millions d'habitants » ; contexte : « capitale... 2,1 millions ». $s_1 \Rightarrow v_1 = 1$, $s_2 \Rightarrow v_2 = 0 \Rightarrow \text{Faithfulness} = 1/2 = 0,5$.

16.2 Answer Relevancy — arête $q \leftrightarrow a$

LLM génère n questions $\{q'_i\}$ depuis la réponse, puis :

$$\text{AnswerRelevancy} = \frac{1}{n} \sum_{i=1}^n \cos(E(q), E(q'_i)).$$

Idée principale

Si la réponse est précise, les questions qu'elle « inspire » ressemblent à q (cosinus élevé) ; vague/hors-sujet $\Rightarrow$ cosinus faible. Mesuré **sans référence**.

16.3 Context Precision — arête $q \leftrightarrow \mathcal{C}$

Pertinence $v_k \in \{0, 1\}$ par chunk ; AP sur la liste :

$$\text{ContextPrecision@K} = \frac{\sum_{k=1}^K (P@k \cdot v_k)}{\sum_{k=1}^K v_k}.$$

Calcul étape par étape

$v = (1, 0, 1)$: $P@1 = 1$, $P@2 = 0,5$, $P@3 = 0,667$.
 $CP = \frac{1 \cdot 1 + 0,5 \cdot 0 + 0,667 \cdot 1}{2} = \frac{1,667}{2} \approx 0,833$. (Récompense les pertinents *placés tôt*.)

16.4 Context Recall — arête $q \leftrightarrow \mathcal{C}$ (avec référence)

Décomposer la référence g en $\{g_1, \dots, g_m\}$, attribution $a_j \in \{0, 1\}$ au contexte :

$$\text{ContextRecall} = \frac{1}{m} \sum_{j=1}^m a_j.$$

Idée principale

Precision : le contexte est-il propre ? **Recall** : est-il complet ? Un recall bas = chunking/retrieval défaillant (l'info n'a pas été récupérée).

16.5 Answer Correctness (avec référence)

Factuel (F1 sur affirmations TP/FP/FN) + sémantique :

$$F1 = \frac{|TP|}{|TP| + \frac{1}{2}(|FP| + |FN|)}, \quad AC = w_1 F1 + w_2 \cos(E(a), E(g)) \quad (w_1=0,75, w_2=0,25).$$

Calcul étape par étape

$|TP| = 2$, $|FP| = 1$, $|FN| = 1$, $\text{sim} = 0,9$: $F1 = \frac{2}{2+1} = 0,667$, $AC = 0,75 \cdot 0,667 + 0,25 \cdot 0,9 = 0,725$.

16.6 Autres & limites

Semantic Similarity = $\cos(E(a), E(g))$; **Context Relevancy** = $\frac{\# \text{phrases pertinentes}}{\# \text{phrases}}$; **Noise sensitivity**, **Aspect Critique**.

Limites

RAGAS dépend d'un LLM-juge $\Rightarrow$ biais et variance. Bonnes pratiques : **temperature=0**, jeu de test suffisant, juge fort. RAGAS évolue : vérifier la version.

16.7 Code : RAGAS de bout en bout

Listing 18 – Generation d'un jeu de test + evaluation RAGAS

```
1 from datasets import Dataset
2 from ragas import evaluate
3 from ragas.metrics import (faithfulness, answer_relevancy,
4                             context_precision, context_recall, answer_correctness)
5
6 samples = {"question": [], "answer": [], "contexts": [], "ground_truth": []}
7 for q, gt in zip(questions, references):
8     ctx = [d.page_content for d in retriever.invoke(q)]
9     ans = chain.invoke(q)
10    samples["question"].append(q); samples["answer"].append(ans)
11    samples["contexts"].append(ctx); samples["ground_truth"].append(gt)
12
13 res = evaluate(Dataset.from_dict(samples), metrics=[
14     faithfulness, answer_relevancy,
15     context_precision, context_recall, answer_correctness])
16 print(res.to_pandas()) # un score par metrique et par question
```

17 Tableau de diagnostic

Symptôme	Diagnostic	Levier
Context recall bas	info non récupérée	chunking, embeddings, k , hybride
Context precision bas	contexte bruité/mal classé	reranking, MMR, filtrage
Faithfulness bas (ctx ok)	le LLM hallucine	prompt, garde-fous, LLM
Answer relevancy bas	réponse hors-sujet/vague	prompt, transform. requête
Answer correctness bas	faux ou incomplet	combinaison des leviers

Idée principale

On lit les métriques **ensemble**. Context recall *haut* mais faithfulness *bas* $\Rightarrow$ bon contexte mal exploité $\Rightarrow$ agir sur la *génération*. C'est ce qui rend l'optimisation **ciblée**.

Partie III — Optimisation

18 Leviers d'optimisation (reliés aux métriques)

On optimise **un levier à la fois**, puis on re-mesure.

Chunking

Pourquoi ça aide

Chunks autonomes $\Rightarrow$ embeddings plus spécifiques $\Rightarrow$ $\uparrow$ context precision *et* recall.

Embeddings (adaptés / fine-tunés)

Pourquoi ça aide

La récupération plafonne à la qualité de l'espace vectoriel $\Rightarrow$ $\uparrow$ recall et precision en amont.

Recherche hybride + RRF

$$\text{score}(d) = \sum_r \frac{1}{\kappa + \text{rang}_r(d)} (\text{dense} + \text{BM25}).$$

Pourquoi ça aide

Dense = sens, sparse = termes exacts ; fusion par rangs $\Rightarrow \uparrow$ context recall robuste.

Listing 19 – Hybride BM25 + dense

```
1 from langchain_community.retrievers import BM25Retriever
2 from langchain.retrievers import EnsembleRetriever
3 bm25 = BM25Retriever.from_documents(chunks); bm25.k = 4
4 hybrid = EnsembleRetriever(
5     retrievers=[bm25, vs.as_retriever(search_kwargs={"k":4})],
6     weights=[0.4, 0.6])
```

MMR (diversité)

$$\text{MMR} = \arg \max_{c \notin S} [\lambda \text{sim}(q, c) - (1 - \lambda) \max_{c' \in S} \text{sim}(c, c')].$$

Pourquoi ça aide

Pénalise la redondance $\Rightarrow$ plus d'info distincte $\Rightarrow \uparrow$ context recall.

Re-ranking (cross-encoder)

Récupérer large (top-50) puis re-classer.

Pourquoi ça aide

Le cross-encoder lit la paire (q, c) ensemble (précis) $\Rightarrow \uparrow$ context precision/NDCG $\Rightarrow$ moins de bruit $\Rightarrow \uparrow$ faithfulness.

Listing 20 – Re-ranking

```
1 from langchain.retrievers import ContextualCompressionRetriever
2 from langchain.retrievers.document_compressors import CrossEncoderReranker
3 from langchain_community.cross_encoders import HuggingFaceCrossEncoder
4 ce = HuggingFaceCrossEncoder(model_name="cross-encoder/ms-marco-MiniLM-L-6-v2")
5 rr = ContextualCompressionRetriever(
6     base_compressor=CrossEncoderReranker(model=ce, top_n=4),
7     base_retriever=vs.as_retriever(search_kwargs={"k": 50}))
```

Transformation de requête (multi-query, HyDE)

Pourquoi ça aide

HyDE comble le fossé « style question / style document » $\Rightarrow \uparrow$ recall sur requêtes difficiles.

Listing 21 – Multi-query

```
1 from langchain.retrievers.multi_query import MultiQueryRetriever
2 mq = MultiQueryRetriever.from_llm(retriever=vs.as_retriever(), llm=llm)
```

Contexte : compression & réordonnement

Pourquoi ça aide

Contre le « lost in the middle » : moins de bruit + pertinent aux extrémités $\Rightarrow \uparrow$ faithfulness.

Prompt et génération

Pourquoi ça aide

Consigne d'ancrage, citations, « je ne sais pas », `temperature=0` $\Rightarrow \uparrow$ faithfulness.

Architectures avancées (parent-doc, CRAG, agentic/Graph RAG)

Pourquoi ça aide

Boucle de vérification / contexte structuré $\Rightarrow$ gains sur les questions multi-sauts.

19 Méthodologie : la boucle fermée

Idée principale

Baseline (mesurer toutes les métriques) $\rightarrow$ identifier le maillon faible (tableau de diagnostic) $\rightarrow$ **changer un seul levier** $\rightarrow$ **re-mesurer** $\rightarrow$ garder si gain. C'est ce qui transforme l'optimisation en démarche *scientifique* : chaque amélioration est prouvée.

20 Synthèse générale

À retenir

- **Pipeline** : indexation (load/split/embed/store) puis récupération + génération ; le LLM est conditionné sur le contexte, formellement par marginalisation sur les k documents.
- **Évaluation** — **toutes les familles** : *récupération* (P/R@ k , MRR, MAP, NDCG), *génération classique* (BLEU, ROUGE, BERTScore), *LLM-juge*, et **RAGAS** (faithfulness, answer relevancy, context precision/recall, answer correctness) comme couche supplémentaire factuelle.
- **Triangle** ($q, \mathcal{C}, a$) : chaque métrique cible une arête $\Rightarrow$ diagnostic puis optimisation ciblée.
- **Optimisation** : chunking, RRF, MMR, reranking, HyDE/multi-query, compression, prompt, CRAG — chacun relié à une métrique, dans une **boucle fermée** baseline $\rightarrow$ levier $\rightarrow$ re-mesure.